

WEEK6 - COS20028 (BIG DATA ARCHITECTURE AND APPLICATION)

Dr Pei-Wei Tsai (Unit Convenor)
Yicun Tian

Week 6 Overview: What You'll Learn

Source	Content
Online Videos (Canvas)	Part 1:Partitioners Part 2:Writing Custom Partitioners Part 3:File Formats
Live Session	Quiz2 Recap List & Hash Map in Java Recap Partitioners
Lab Activities	1.Costomies Partitioners 2.Custom Writable Comparable 3. Sequence Files and File Compression



RECAP – LIST

Understanding **List** in Java

- **Definition:**

In Java, a List is an ordered collection used to store a group of elements, where duplicates are allowed.

- **Key Characteristics:**

- **Ordered:** Maintains the order of insertion.
- **Allows duplicates:** You can store the same value multiple times.
- **Index-based access:** Elements can be accessed using their index (starting from 0).
- **Subtype of Collection:** It extends the Collection interface.

- **Common **implementations**: **ArrayList** and **LinkedList**.**

- **Example Scenarios:**

- Storing a list of students: ["Alice", "Bob", "Charlie"]
- Items in a user's shopping cart
- Weekly task list

Common Methods to Access and Modify a List

1. **get(index)** – Returns the element at the specified position in the list.
2. **size()** – Returns the number of elements in the list.
3. **contains(element)** – Checks if the list contains the specified element.
4. **indexOf(element)** – Returns the index of the first occurrence of the element, or -1 if not found.
5. **isEmpty()** – Returns true if the list contains no elements.
6. **add(element)** – Adds an element to the end of the list.
7. **remove(index)** – Removes the element at the specified index.
8. **remove(element)** – Removes the first occurrence of the specified element.

Example

```
// indexOf(): find the first index of "Alice"  
System.out.println("Index of 'Alice': " + students.indexOf("Alice"));  
// Output: 0
```

```
// remove(index): remove element at index 2 ("Charlie")  
students.remove(2);  
System.out.println("After removing index 2: " + students);  
// Output: [Alice, Bob, Alice]
```

```
// remove(element): remove the first occurrence of "Alice"  
students.remove("Alice");  
System.out.println("After removing 'Alice': " + students);  
// Output: [Bob, Alice]  
// Output: true
```

```
import java.util.ArrayList;  
import java.util.List;  
  
public class ListExample {  
    public static void main(String[] args) {  
        // Create a List of strings  
        List<String> students = new ArrayList<>();  
  
        // Add elements to the list  
        students.add("Alice");  
        students.add("Bob");  
        students.add("Charlie");  
        students.add("Alice"); // duplicate value
```

```
        students.remove("Charlie"))
```

List- T/F

- 1) A List in Java maintains the order of insertion.
- 2) A List does not allow duplicate elements.
- 3) You can access elements in a List using their index, starting from 0.
- 4) The size() method returns the first element of the List.
- 5) Both ArrayList and LinkedList are common implementations of the List interface.
- 6) The isEmpty() method returns true only when the List has at least one element.

List- T/F

1) A List in Java maintains the order of insertion. (True)

2) A List does not allow duplicate elements. (False)

Explanation: A List does allow duplicates. For example, ["Alice", "Bob", "Alice"] is valid, and both "Alice" values are kept.

3) You can access elements in a List using their index, starting from 0. (True)

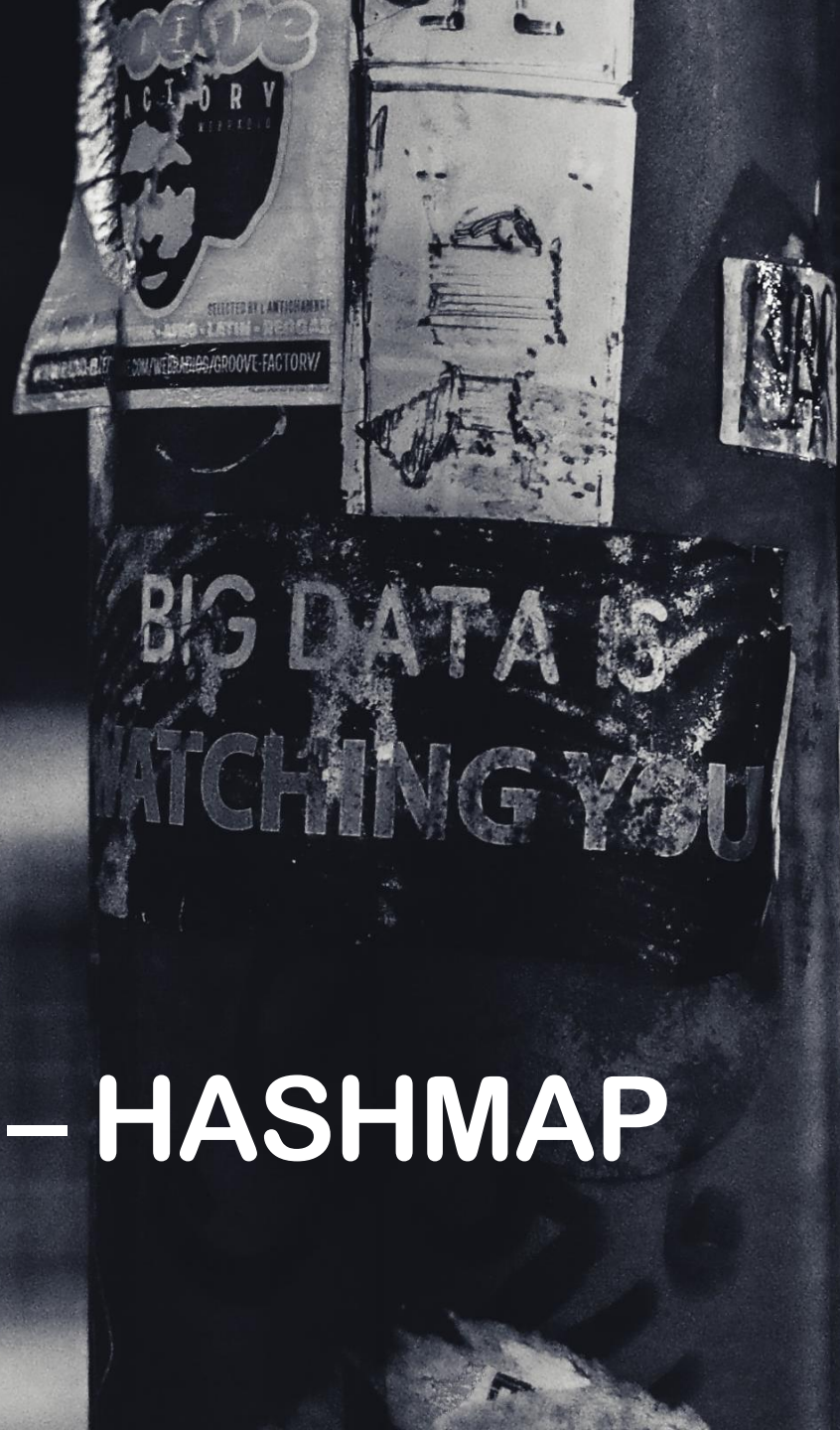
4) The size() method returns the first element of the List. (False)

Explanation: size() does not return an element. It returns the number of elements in the list. To get the first element, you would use get(0).

5) Both ArrayList and LinkedList are common implementations of the List interface. (True)

6) The isEmpty() method returns true only when the List has at least one element. (False)

Explanation: isEmpty() returns true only if the List has zero elements. If the list has at least one element, it will return false.



RECAP – HASHMAP

Understanding HashMap in Java

- HashMap is a key-value pair data structure in Java
- Data is stored in the format: Key → Value
- Keys must be **unique**, but values can be duplicated
- Offers high lookup efficiency, suitable for fast data retrieval (based on a hash table)
- Does not guarantee ordering — insertion and output order may differ
- Allows null as a key (only one) and as a value
- Commonly used for mapping relationships, e.g., ppl → Phone Number, Year → ID

ID	Name
20230001	Alice
20230002	Bob
20230003	Charlie

HashMap<Integer, String>

Commonly Used HashMap Methods and Their Purpose

- 1) **put(K key, V value)**: Adds or updates a key-value pair.
- 2) **get(Object key)**: Retrieves the value associated with the specified key.
- 3) **getOrDefault(Object key, V defaultValue)**: Returns the value for the given key, or a default value if the key is not found.
- 4) **containsKey(Object key)**: Checks whether the map contains the specified key.
- 5) **remove(Object key)**: Removes the key-value pair associated with the specified key.
- 6) **keySet()**: Returns a set view of all keys in the map.
- 7) **values()**: Returns a collection view of all values in the map.
- 8) **size()**: Returns the number of key-value pairs currently in the map.
- 9) **isEmpty()**: Checks if the map is empty (i.e., contains no entries).
- 10) **clear()**: Removes all key-value pairs from the map.

+-----+-----+	
ID	Name
+-----+-----+	
20230001	Alice
20230002	Bob
20230003	Charlie
+-----+-----+	

Example

```
import java.util.HashMap;

public class HashMapDemo {
    public static void main(String[] args) {
        // Create a HashMap and add key-value pairs
        HashMap<Integer, String> studentMap = new HashMap<>();
        studentMap.put(20230001, "Alice");
        studentMap.put(20230002, "Bob");
        studentMap.put(20230003, "Charlie");
    }
}
```

```
// get(): Retrieve the value associated with the given key
System.out.println("get(20230001): " + studentMap.get(20230001));
// Output: Alice
```

```
// remove(): Remove the key-value pair for a given key
studentMap.remove(20230003);
System.out.println("get(20230003): " + studentMap.get(20230003));
// Output: null (since it was removed)
```


Difference Between Map and HashMap

Map is the specification, HashMap is the implementation.

You can think of Map as the parent and HashMap as the child — the child does the actual work, but using the parent's name gives you more flexibility.

Comparison Item	Map	HashMap
Type	An interface	A concrete class that implements Map
Can create object directly?	No (new Map<>() is invalid)	Yes, you can use new HashMap<>()
Purpose	Defines the contract (methods like put, get, remove)	Provides the actual implementation (based on hash table)

HashMap-T/F

- 1) You can directly create an object of type Map using new Map<>().
- 2) HashMap allows one null key and multiple null values.
- 3) The put() method in HashMap is used to retrieve the value associated with a key.
- 4) HashMap maintains the order in which entries were inserted.
- 5) Using getOrDefault(key, defaultValue) can prevent null results if a key doesn't exist.
- 6) HashMap is one of several classes that implement the Map interface.

HashMap-T/F

You can directly create an object of type Map using new Map<>().

✗ False Explanation: Map is an interface and cannot be instantiated directly.

HashMap allows one null key and multiple null values. 

✓ True Explanation: HashMap permits one null key and multiple null values.

The put() method in HashMap is used to retrieve the value associated with a key.

✗ False Explanation: put() adds or updates key-value pairs; get() is used to retrieve values. 

HashMap maintains the order in which entries were inserted.

✗ False Explanation: HashMap does not guarantee insertion order.

Using getOrDefault(key, defaultValue) can prevent null results if a key doesn't exist.

✓ True Explanation: It returns the default value instead of null when the key is not found.

HashMap is one of several classes that implement the Map interface.

✓ True Explanation: Other implementations include TreeMap, LinkedHashMap, etc.

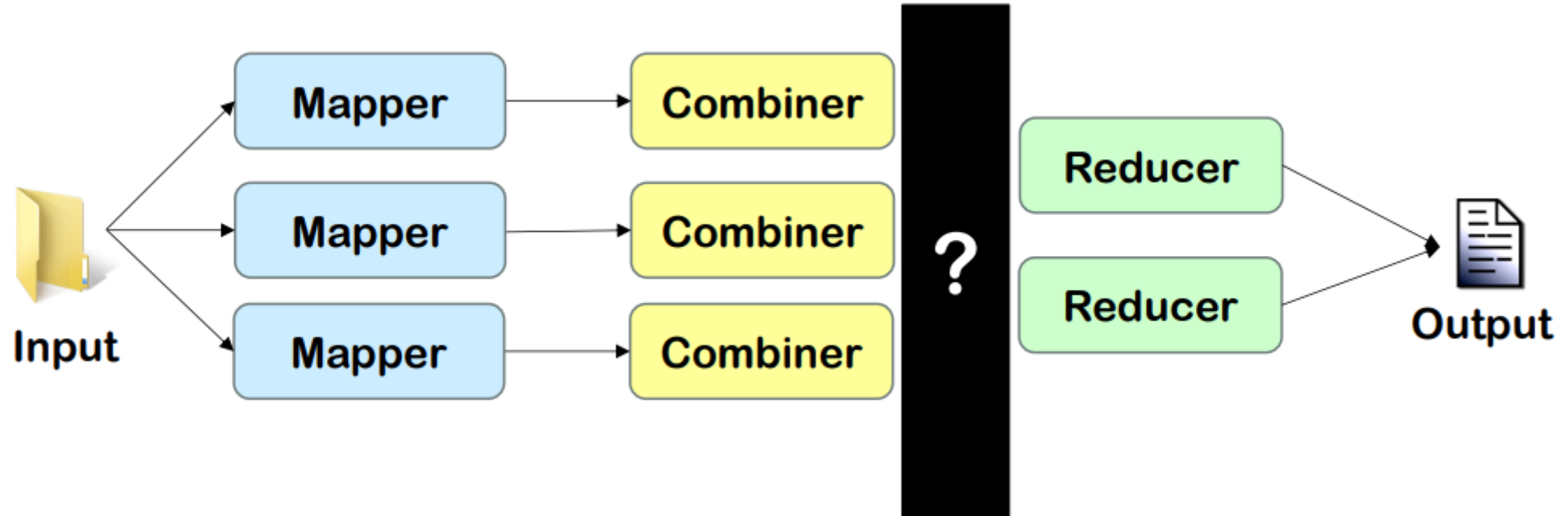


BIG DATA IS
WATCHING YOU

RECAP – PARTITIONERS

Bridge

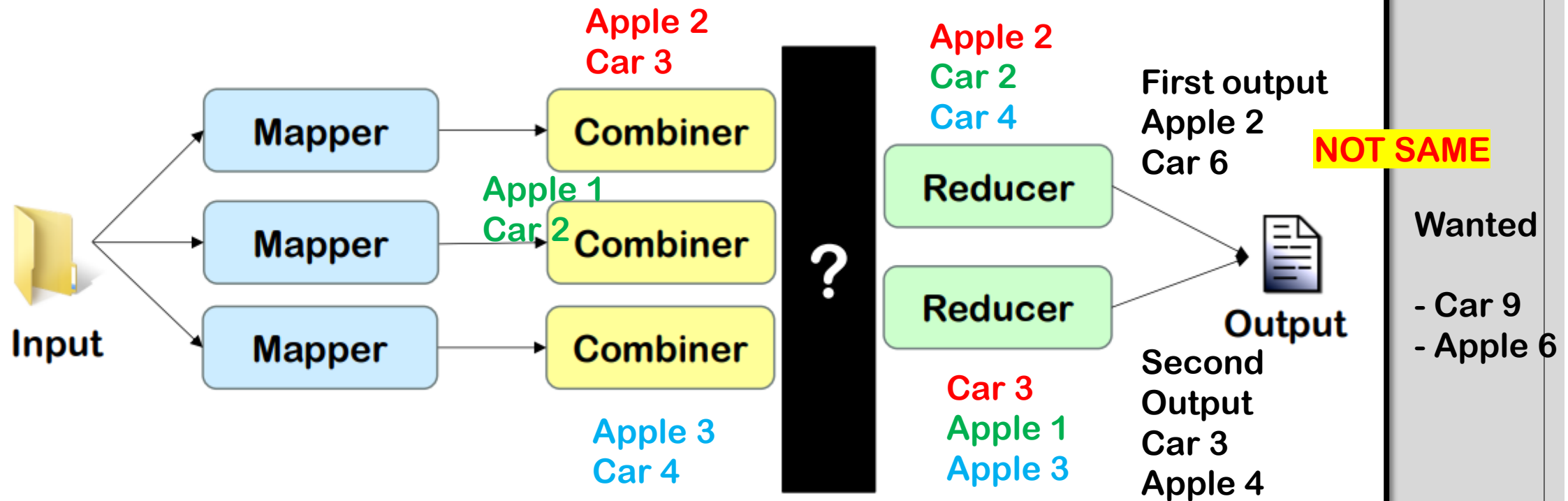
What do you think this black box is doing?
This black box is responsible for deciding where each key-value pair should go.



Bridge

Why can't we just **randomly** send the data to any Reducer?

If the same key ends up going to **different Reducers**, then each Reducer would only have **part of the data** for that key — and it wouldn't be able to do the final aggregation correctly.

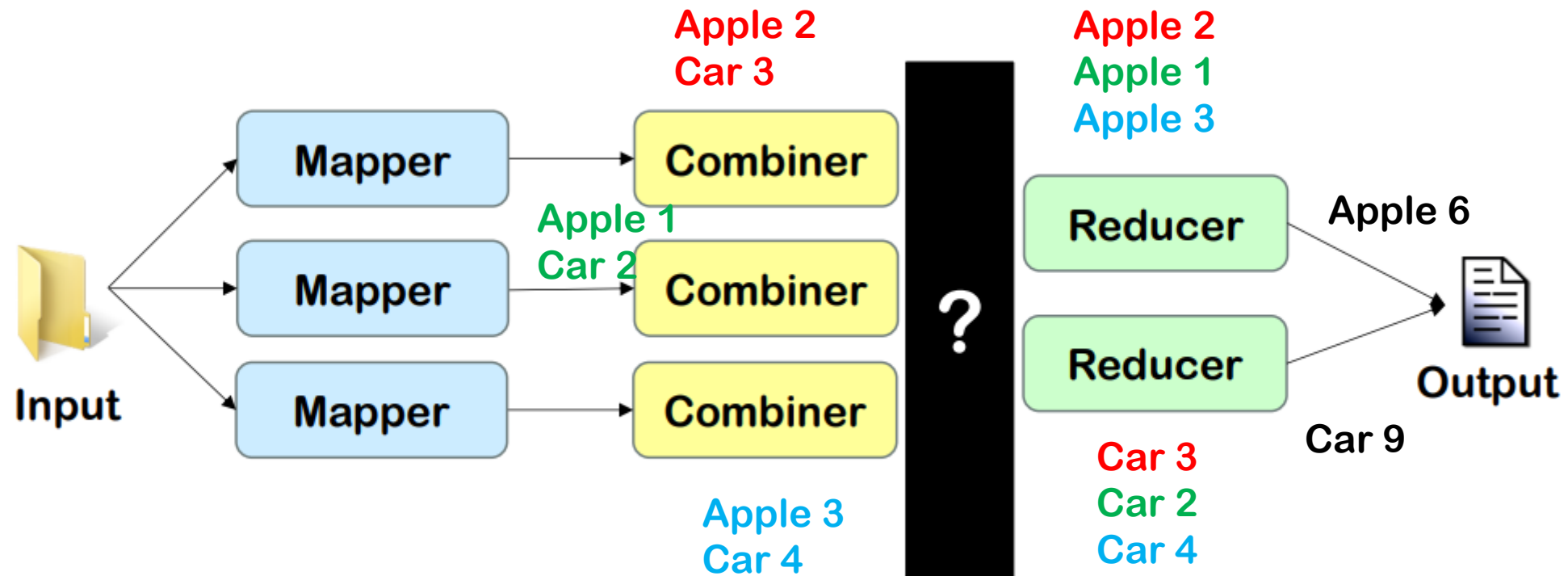


Bridge

How do the intermediate key-value pairs know which Reducer to go to?

For example, it might send all keys that are 'apple' to Reducer 1, and the rest with keys like 'car' to Reducer 2.

ensures that all values associated with the same key go to the same Reducer.



What You Learned from the MapReduce Pipeline Slides

MapReduce Framework consists of:

Input → Mapper → (Optional) Combiner → Partitioner → Reducer → Output

- Mappers work in parallel to process input splits.
- Combiners perform local aggregation to reduce data transferred to reducers.
- Partitioner determines which Reducer receives each key-value pair.
- Reducers aggregate values for each key and produce final output.
- The number of Mappers and Reducers can be adjusted for performance and scalability.

Default Partitioner - HashPartitioner

- The default Partitioner in MapReduce is called HashPartitioner.
- **$\text{partition} = (\text{key.hashCode()} \& \text{Integer.MAX_VALUE}) \% \text{numReducers}$**
- All key-value pairs with the same key will always go to the same Reducer.
- The hash output is consistent: the same input key always produces the same hash.

Final assignment result:

Step 1: Call

Let's assume

- "apple" ("apple", 1), ("apple", 1), ("car", 1), ("banana", 1), ("banana", 1)
- "banana"
- "car".h
- "dog".h ("dog", 1)

- Reducer 0 receives:

- Reducer 1 receives:

$(\text{hashCode} \% \text{numReducers})$

r 0 and Reducer 1.

Sent to Reducer 0

Sent to Reducer 0

▷ Reducer 0

▷ Reducer 1

How Many Reducers Do You Need?

Type	Characteristics	Suitable Scenarios
Single Reducer	Sorted output, globally ordered, easy to manage	When output must be a single file, or global sorting is needed
Fixed Number Reducers	Clearly defined grouping	Grouping by month, by weekday, etc.
Variable Number Reducers	Developer-defined, load balancing	Most general data processing tasks



RECAP – CUSTOMISED PARTITIONERS

Why Use a Custom Partitioner?

- **For Secondary Sort**
 - When your key is a composite pair like (a, b), and you want to partition by a, then sort by b within each reducer.
 - In this case, all records with the same a value must be sent to the same Reducer.
- **To Avoid Performance Bottlenecks**
 - The default partitioning may send too much data to a single reducer, while others remain underutilized.
 - A custom partitioner helps distribute the workload more evenly and avoids having one reducer become a performance bottleneck.

DRIVER CODE

```
// Defining the input/output paths.  
FileInputFormat.setInputPaths(job, new Path(args[0]));  
FileOutputFormat.setOutputPath(job, new Path(args[1]));  
  
// Set the Mapper and the Reducer.  
job.setMapperClass(LogFileMapper.class);  
job.setReducerClass(LogFileReducer.class);  
job.setPartitionerClass(YearPartitioner.class);  
job.setNumReduceTasks(3);  
  
// Intermediate output key/value produced by Mapper.  
job.setMapOutputKeyClass(Text.class);  
job.setMapOutputValueClass(IntWritable.class);
```


Custom Partitioner

```
import org.apache.hadoop.mapreduce.Partitioner;

public class YearPartitioner<K2, V2> extends Partitioner<Text, Text> implements
    Configurable {

    private int reducer;

    public int getPartition(Text key, Text value, int numReduceTasks) {
        // determine reducer number between 0 and numReduceTasks-1
        //...
        return reducer;
    }
}
```

Example

- **Scenario: Partition Data by Year**
- **You want to send all data from before 2000 to Reducer 0, and all data from 2000 and after to Reducer 1.**
- **Let's say your mapper emits data like this:**
 - ("1998", "value1")
 - ("2001", "value2")
 - ("1995", "value3")
 - ("2023", "value4")

How to Set This Partitioner in Your Driver Code

- Goal: You want to send all data from before 2000 to Reducer 0, and all data from 2000 and after to Reducer 1.
- In your Job configuration:
- `job.setPartitionerClass(YearPartitioner.class);`
- `job.setNumReduceTasks(2);`

Custom Partitioner Class

```
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Partitioner;

public class YearPartitioner extends Partitioner<Text, Text> {

    @Override
    public int getPartition(Text key, Text value, int numReduceTasks) {
        // Parse year from key
        int year = Integer.parseInt(key.toString());

        // Divide into two groups: before 2000 → Reducer 0, 2000 and after → Reducer 1
        if (year < 2000) {
            return 0 % numReduceTasks;
        } else {
            return 1 % numReduceTasks;
        }
    }
}
```

Goal: You want to send all data from before 2000 to Reducer 0, and all data from 2000 and after to Reducer 1.

Key	Value	Sent to Reducer
1998	value1	Reducer 0
2001	value2	Reducer 1
1995	value3	Reducer 0
2023	value4	Reducer 1

Example2

- You want to assign records to different reducers based on the day of the week.
- For example:
 - Records from Monday and Wednesday go to Reducer 0
 - Records from Tuesday and Friday go to Reducer 1
 - All other days go to Reducer 2

Custom Partitioner Class

```
@Override
public void setConf(Configuration conf) {
    this.conf = conf;

    // Read configured weekdays from job configuration
    String r0 = conf.get("reducer0.days", "Monday,Wednesday");
    String r1 = conf.get("reducer1.days", "Tuesday,Friday");

    reducer0Days.addAll(Arrays.asList(r0.split(",")));
    reducer1Days.addAll(Arrays.asList(r1.split(",")));
}
```

```
@Override
public int getPartition(Text key, Text value, int numReduceTasks) {
    String day = key.toString().trim();

    if (reducer0Days.contains(day)) {
        return 0 % numReduceTasks;
    } else if (reducer1Days.contains(day)) {
        return 1 % numReduceTasks;
    } else {
        return 2 % numReduceTasks;
    }
}
```